

Project APEX Research Reading and Design Guide

From paper diagrams to falsifiable F1 world-model experiments

OpenAI

August 2026

Table of Contents

1 How to read a world-model paper as an engineer

A research paper is not a tutorial, a repository is not a proof, and a benchmark result is not automatically relevant to F1 telemetry. The job of this guide is to turn each paper into six concrete objects:

Problem claim — what limitation the paper says exists.

Mechanism — the smallest architectural or objective change that addresses it.

Training contract — what data, targets, losses and optimization are required.

Evaluation claim — what evidence the authors provide and what they do not provide.

Reproduction slice — the smallest runnable implementation that isolates the mechanism.

APEX experiment — a matched ablation whose result could change an engineering decision.

Before reading equations, write down the interfaces:

observation o_t

state / belief s_t

intervention a_t

context c_t

reward or task target r_t

next observation / representation

Then label what is available during training, posterior inference, open-loop imagination and deployment. Most accidental misunderstandings come from letting a training-only signal leak into imagination.

1.1 The three-pass method

1.1.1 Pass 1: Claim map

Read title, abstract, introduction, figures and conclusion. Do not stop to decode every symbol. Write one sentence for the problem, one for the mechanism and one for the result. If you cannot, the paper has not yet become a mental model.

1.1.2 Pass 2: Execution map

Trace one batch through the method. List tensor shapes, gradient paths and stop-gradient paths. Mark which network is updated by which loss. Mark any target network, replay buffer, latent sample or normalization rule.

1.1.3 Pass 3: Evidence map

For every table or curve, ask:

What changed between control and treatment?

What remained fixed?

How many seeds or tasks support the claim?

Does the metric test the claimed mechanism?

What failure is hidden by the average?

Which assumption changes in APEX?

Finish by writing a reproduction test that could fail. “Implement Dreamer” is not a test. “Under the same session split and parameter budget, does a stochastic RSSM improve calibrated 20-step telemetry coverage over a deterministic GRU?” is testable.

2 DreamerV3: from latent dynamics to imagined behaviour

2.1 The problem it addresses

Model-free reinforcement learning learns behaviour directly from real interaction and can be data hungry. Dreamer learns a world model from experience, then improves behaviour using trajectories generated inside that model. DreamerV3 focuses on one configuration that performs across varied domains without per-domain tuning.

The system is not merely an RSSM. It is a collection of contracts:

experience replay

↓

encoder + recurrent state-space model

↓

observation / reward / continuation predictions

↓

latent imagined trajectories

↓

actor and critic learning

If you implement only a stochastic recurrent transition, you have built one component, not the full algorithm.

2.2 Mechanism map

The recurrent state combines deterministic memory with stochastic latent state. During representation learning, current observations inform a posterior latent. A prior predicts the latent without seeing the

observation. Reconstruction/prediction losses preserve useful evidence; KL-related terms align posterior and prior enough for imagination.

During imagination, the model receives latent state and actions but no future observation. Reward and continuation heads score imagined outcomes. Actor and critic updates occur on those imagined trajectories.

2.3 Paper-to-code questions

Before implementing, answer:

Which tensors are detached during actor/critic updates?

Does the actor receive stochastic samples, distribution statistics or both?

How are returns normalized or transformed?

How are reward scales handled across domains?

How is the KL objective balanced to prevent collapse or an unusable prior?

What is the imagination horizon and how does it interact with model error?

2.4 Minimal reproduction slice

Do not begin with an F1 agent. Build a stochastic one-dimensional car where hidden grip changes between episodes. Train:

Deterministic GRU transition.

RSSM with posterior/prior and state decoder.

Reward head for progress minus instability.

CEM planner using the RSSM.

Only then, a small actor trained on imagined trajectories.

Your first milestone is not reward. It is showing that prior rollouts widen uncertainty when grip is ambiguous and that the posterior narrows after evidence.

2.5 APEX experiment

Question: Does stochastic latent belief improve decisions under hidden grip compared with deterministic recurrence?

Control: GRU with matched hidden/parameter budget.

Treatment: RSSM with deterministic memory, stochastic latent and calibrated prediction intervals.

Held fixed: canonical data, session split, history, horizon, optimizer budget, scenario controls and evaluation.

Primary metric: negative log likelihood or interval coverage by horizon under hidden-grip sessions.

Guardrails: mean speed MAE, physical violations, latency, prior-posterior gap and calibration.

Decision rule: keep RSSM only if uncertainty improves decision safety or planning under ambiguity without unacceptable degradation in mean trajectory.

3 Mamba and selective state-space models

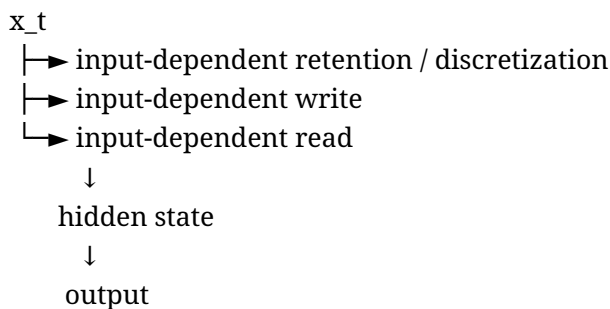
3.1 The problem it addresses

Transformers use content-dependent attention but attention cost grows quadratically with sequence length. Structured state-space models process long sequences efficiently but traditional linear time-invariant dynamics have limited content-dependent selection. Mamba introduces input-dependent selective state-space parameters and an implementation designed for modern hardware.

The transferable insight is not “Mamba is faster.” It is that a sequence token can determine what enters memory, what persists and what is read, while retaining a recurrent/state-space computation.

3.2 Mechanism map

A conceptual selective recurrence is:



The complete Mamba block contains more than a gate. Its empirical and throughput claims rely on parameterization, projections, local mixing and a selective-scan implementation. A custom $a(x)h + (1-a(x))b(x)$ cell is a teaching model, not an equivalent reproduction.

3.3 Paper-to-code questions

Which parameters depend on input and which remain global?

How is stability enforced or encouraged?

What is computed recurrently during inference?

What is parallelized during training?

What sequence lengths and hardware are required for throughput advantage?

Are gains caused by selection, parameter count, training recipe or implementation?

3.4 Minimal reproduction slice

Create a delayed-event task. Most frames are routine; one rare token determines a target 200 steps later. Compare:

Fixed linear SSM.

GRU.

Toy selective SSM.

Official Mamba implementation, only after the first three are understood.

Plot retention/write behaviour around the rare token. Match parameter counts and training steps.

Measure memory accuracy and wall-clock throughput separately.

3.5 APEX experiment

Question: Does input-dependent memory improve long-horizon telemetry representation when rare regime changes matter?

Use histories long enough to create a real bottleneck. Test sessions with safety-car changes, rain onset or sustained tyre degradation. Compare speed/latency against GRU and temporal convolution. A short 32-frame window is unlikely to justify architectural migration by itself.

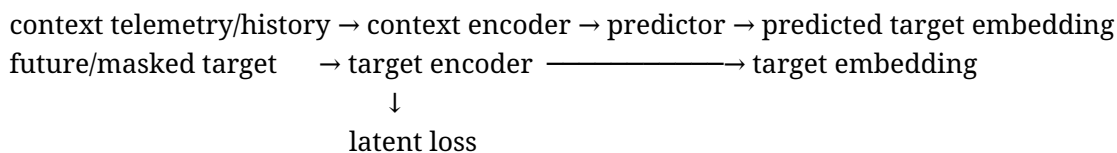
4 I-JEPA and predictive representations

4.1 The problem it addresses

Reconstructive self-supervision asks a model to reproduce every raw detail. A joint-embedding predictive architecture predicts representations of missing/target regions from context. The target representation can discard unpredictable local detail while preserving semantic structure.

For telemetry, the analogue is predicting a future or masked-state representation instead of every sensor value. This can be beneficial if sensor noise and redundant channels dominate reconstruction. It can be harmful if the target encoder discards rare transition cues.

4.2 Mechanism map



The asymmetry between context and target branches, stop-gradient/target updates, masking and representation regularity prevent trivial collapse.

4.3 Diagnostics before celebration

A low latent loss is meaningless without:

Per-dimension variance.

Embedding covariance/effective rank.

Nearest-neighbour diversity.

Linear probes for physical variables.

Future-event retrieval.

Downstream rollout or decision value.

Checks that masked information cannot be copied through shortcuts.

4.4 APEX experiment

Pretrain an encoder on unlabeled canonical sessions by predicting future target embeddings. Freeze it and train a small dynamics head. Compare with:

Random frozen encoder.

Autoencoder-pretrained encoder.

End-to-end supervised GRU.

JEPA-style pretrained encoder plus head.

Report data efficiency curves—not only final performance. The strongest transfer claim is improved performance with fewer labelled/high-quality sessions under the same held-out events.

5 LeWorldModel and efficient planning-oriented latent models

5.1 Reading discipline

When a paper claims faster planning, separate:

Latent transition speed.

Number of model evaluations required by the planner.

Representation/encoder cost.

Training cost.

Hardware and batch size.

Quality of imagined trajectories.

Task success under equal wall-clock budget.

A model can be faster per step but require more samples, or produce lower-quality rollouts that mislead planning. Planning speed and decision quality must be reported together.

5.2 APEX reproduction slice

Use the same CEM planner with two dynamics backends:

Deterministic GRU latent/state transition.

Alternative efficient latent transition inspired by the paper.

Hold candidate count, horizon, reward and constraints fixed. Measure:

Rollouts per second.

Best score found under fixed wall-clock time.

Real/synthetic-environment outcome of the selected action.

Model exploitation rate.

Memory and startup cost.

The project should adopt the method only if planning throughput improves while selected actions remain valid under a trusted evaluator.

6 Fine-tuning an existing world or sequence model

Fine-tuning begins with interface compatibility, not checkpoint popularity.

6.1 Compatibility audit

Question

Why it matters

What was the observation type?

Pixels, tokens and telemetry have different invariances.

What actions were used?

A latent trained without interventions may not support counterfactual control.

What time step and horizon?

Dynamics meaning changes with sample rate.

What normalization and units?

Parameter scales encode the source representation.

What objective shaped the latent?

Reconstruction, contrastive, predictive and control objectives preserve different information.

What domain diversity?

Narrow pretraining may transfer less than a small domain model.

Is the implementation/license operationally usable?

Research value and deployability differ.

6.2 Adaption ladder

Frozen features + linear probe. Cheapest test of accessible information.

Frozen backbone + nonlinear head. Tests whether representation is useful but readout is nonlinear.

Parameter-efficient adapters or selected-layer tuning. Adapts domain while preserving most weights.

Gradual unfreezing. Expands capacity only when validation evidence supports it.

Full fine-tuning. Highest flexibility and highest overfitting/forgetting risk.

Train from scratch baseline. Always retain this control for compact telemetry models.

Track representational drift, source-task regression when relevant, and performance by data volume. Fine-tuning is justified by better data efficiency, final performance or operational capability—not by using a larger name.

7 Designing your own APEX world model

An original architecture begins with a measured failure.

7.1 Failure-to-mechanism worksheet

Measured failure

Candidate mechanism

Simpler alternative to rule out

Long history forgotten

Structured/selective memory

Longer window, GRU tuning, temporal convolution

Multi-modal future

Stochastic latent or mixture head

Ensemble, quantile regression

Poor unseen-track transfer

Track representation / equivariance

Better split, geometry features, augmentation

Planner exploitation

Uncertainty/conservative objective

Hard constraints, OOD action limits

Slow planning

Efficient latent transition

Batching, smaller model, fewer candidates

Rare events erased

Event-aware objective/memory

Reweighting, targeted windows

7.2 Architecture proposal template

Write the following before code:

Observed failure: metric, horizon, subgroup and examples.

Hypothesis: why the current mechanism fails.

New mechanism: exact information path being added or changed.

Control: strongest simpler model.

Matched budget: parameters, data, steps and compute.

Primary evidence: metric that directly tests the mechanism.

Guardrails: validity, latency, calibration and baseline regression.

Ablations: remove each new component independently.

Failure/rollback condition: result that causes rejection.

Reproduction package: code, config, data hash, seeds and report.

7.3 Example original direction: geometry-conditioned selective RSSM

A possible research model could combine:

Deterministic recurrent belief.

Stochastic latent for unobserved grip.

Track-geometry encoder.

Input-dependent selective memory rates conditioned on curvature/regime.

Prior imagination under future controls.

Calibrated state distribution and physical validity penalties.

This sounds impressive, but it contains several hypotheses. Test them separately:

Does geometry conditioning improve unseen-track transfer?

Does stochastic latent improve uncertainty under hidden grip?

Does selection improve retention of rare regime changes?

Does combining them produce gains beyond each component?

Only the ablation matrix turns the architecture into research.

8 Paper reading exercises

For each paper:

Draw the training-time graph and imagination-time graph separately.

Mark every gradient and stop-gradient path.

List every distribution and what conditions it sees.

Reproduce the smallest mechanism on a toy sequence.

Design a failure case where the mechanism should help.

Design a case where it should not help.

State the APEX adaptation and the assumption it changes.

Write a matched ablation and rejection criterion.

9 Primary source list

DreamerV3: [arXiv:2301.04104](#)

Mamba: [arXiv:2312.00752](#)

I-JEPA: [arXiv:2301.08243](#) and the official implementation

LeWorldModel: primary paper and official implementation

FastF1 official documentation

OpenF1 official documentation

PyTorch official documentation

EA F1 25 Data Output Specification